

DIGITAL LOGIC

Chapter 7 Memory

2025 Fall

This PowerPoint is for internal use only at Southern University of Science and Technology.
Please do not repost it on other platforms without permission from the instructor.

Today's Agenda

- Recap
 - Asynchronous Counter
 - Synchronous Counter
 - Binary counter
 - Ring counter
 - Johnson counter
 - Design a synchronous counter
- Context
 - RAM
 - ROM
 - PLA
 - FPGA
- Reading: Textbook, Chapter 7

Outline

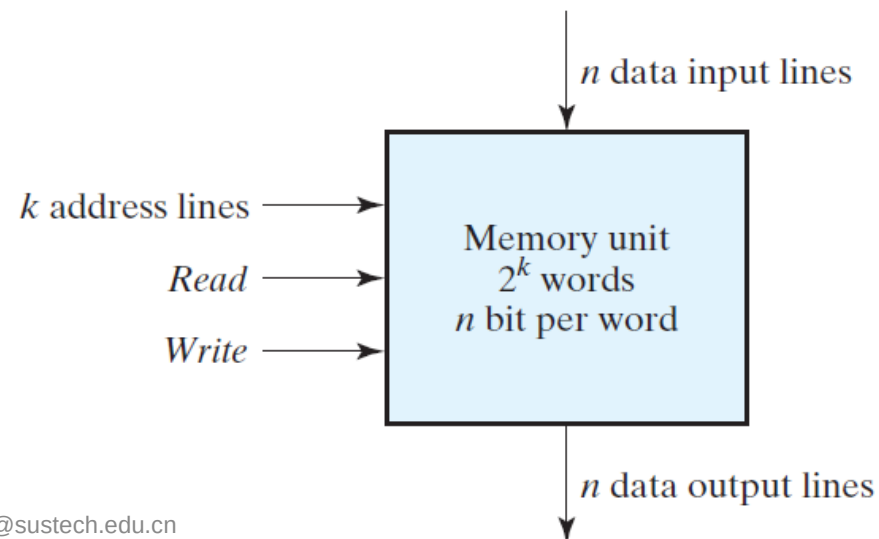
- Random-Access Memory
- Memory Decoding
- Read-Only Memory
- Programmable Logic Array
- Sequential Programmable Devices-FPGA

Memory Unit

- A collection of storage cells together with associated circuits needed to transfer information in and out of storage
- RAM: Random-Access Memory
 - Volatile (memory units that lose stored information when power is turned off)
 - To accept new information for storage to be available later for use
 - read/write operation
- ROM: Read-Only Memory
 - Nonvolatile
 - The information inside can not be altered by writing
 - Programmable devices are specified by some hardware procedure

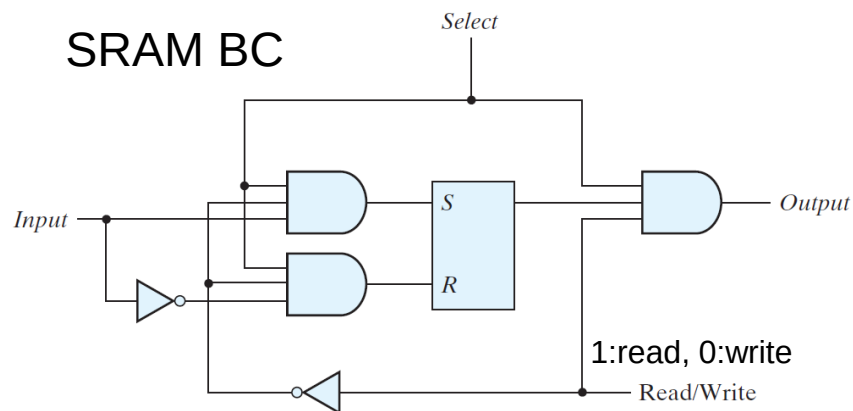
Random-Access Memory

- Characteristics
 - The time it takes to transfer data to or from any desired location is always the same.
 - A memory unit stored binary information in groups of bits (words)
 - The size of the RAM is $2^k \times n$ bits. It has k address lines, n input data lines and n output data lines.
 - For a commodity RAM, $n=8, 16, 32$ or 64 .
- The communication between a memory and its environment is achieved through
 - Data input lines
 - Data output lines
 - Address selection lines
 - Control lines (read and write)



Memory Content Array & Memory Cell

- A memory with k -bit address has 2^k words (depth)
 - (depth)x(width) memory
 - eg. 1K x 16 memory (1024x16 memory)
- A memory cell is also called a binary storage cell (BC)



Memory address		Memory content
Binary	Decimal	
0000000000	0	1011010101011101
0000000001	1	1010101110001001
0000000010	2	0000110101000110
	⋮	⋮
1111111101	1021	1001110100010100
1111111110	1022 2^k-2	0000110100011110
1111111111	1023 2^k-1	1101111000100101

k-bit address **n bits data**

Static RAM vs. Dynamic RAM

- A memory cell (MC) can be considered as a clocked D latch with an AND gate and an output driver
- For a **static RAM (SRAM)**, MC is constructed by 6 transistors
 - using cross-coupled inverters to serve as a latch
 - and implementing the input AND gate and the output driver with one transistor each.
- For a **dynamic RAM (DRAM)**, MC is constructed by only 1 transistor
 - The latch is implemented by a capacitor.
 - It needs to be refreshed periodically (read and write back).
 - It has high density (therefore low cost)

Write and Read Operation

- Write operation
 - Apply the binary address to the address lines
 - Apply the data bits to the data input lines
 - Activate the write input (0)
- Read operation
 - Apply the binary address to the address lines
 - Activate the read input (1)

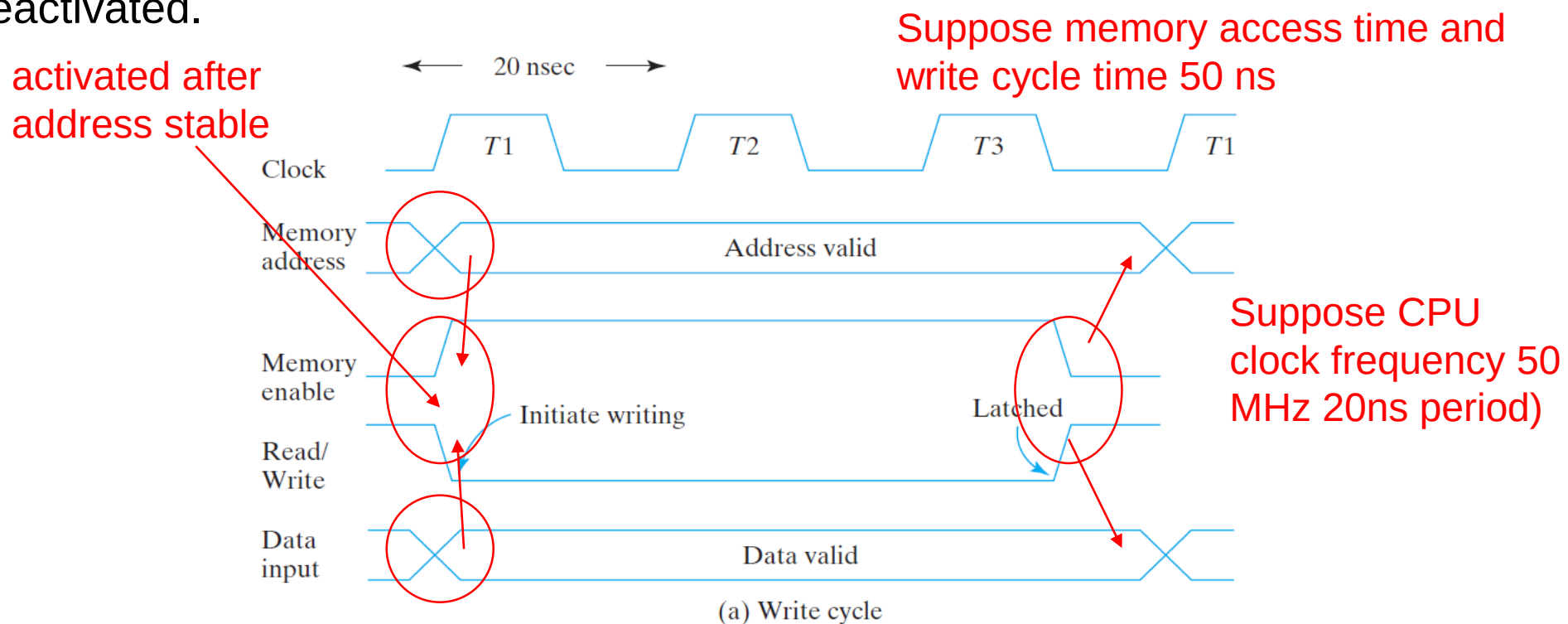
Memory Enable	Read/Write	Memory Operation
0	X	None
1	0	Write to selected word
1	1	Read from selected word

Timing Waveforms

- The operation of the memory unit is controlled by an external device such as a CPU
- The access time is the time required to select a word and read it
- The write cycle time is the time required to complete a write operation
- Read and write operations must be controlled by CPU and be synchronized with an external clock.

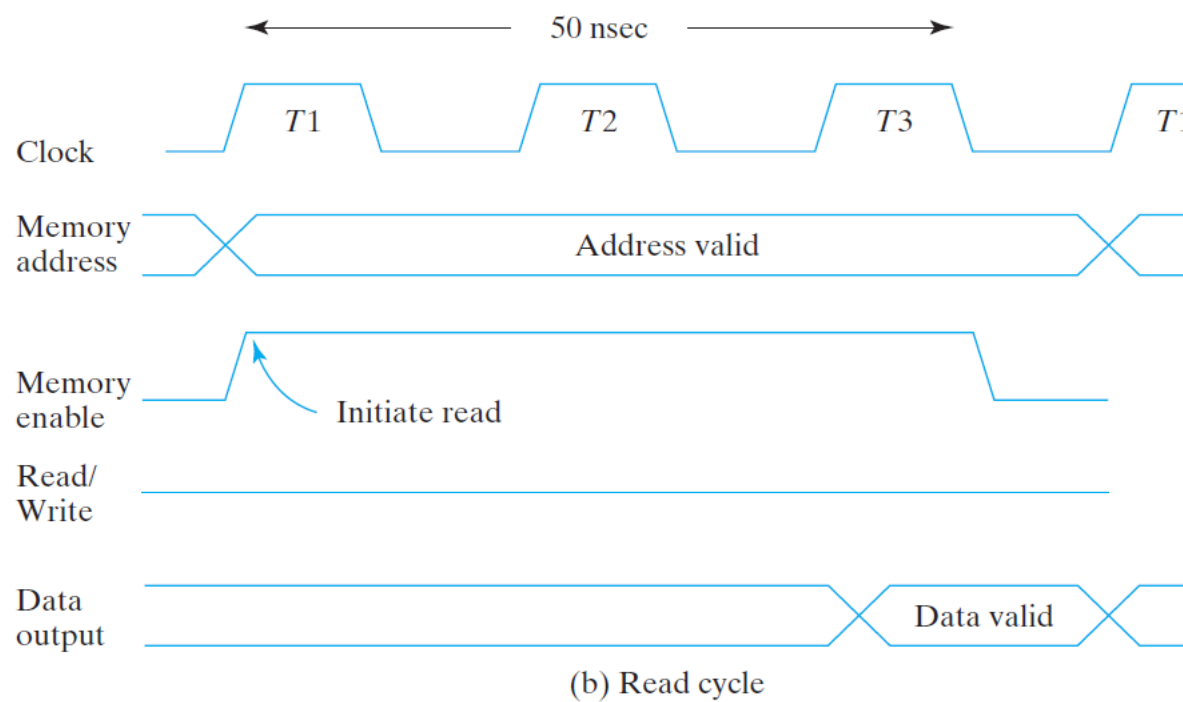
Timing Waveforms

- A write cycle
 - T1: providing the address and input data to the memory, and the write/read control signal
 - Address bus and read/write control must stay active for 50ns
 - The address and data signal must remain stable for a short time after control signal is deactivated.



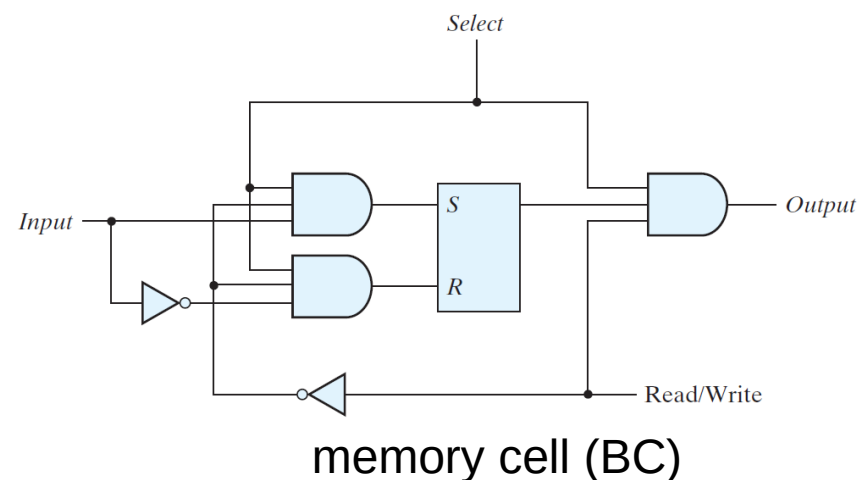
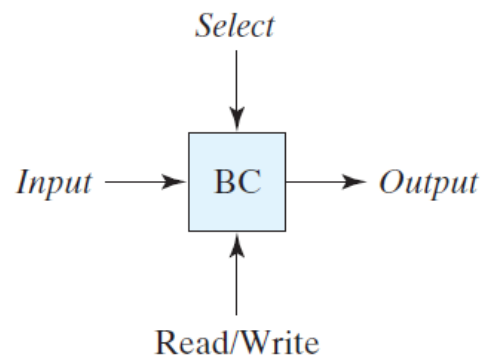
Timing Waveforms

- A read cycle



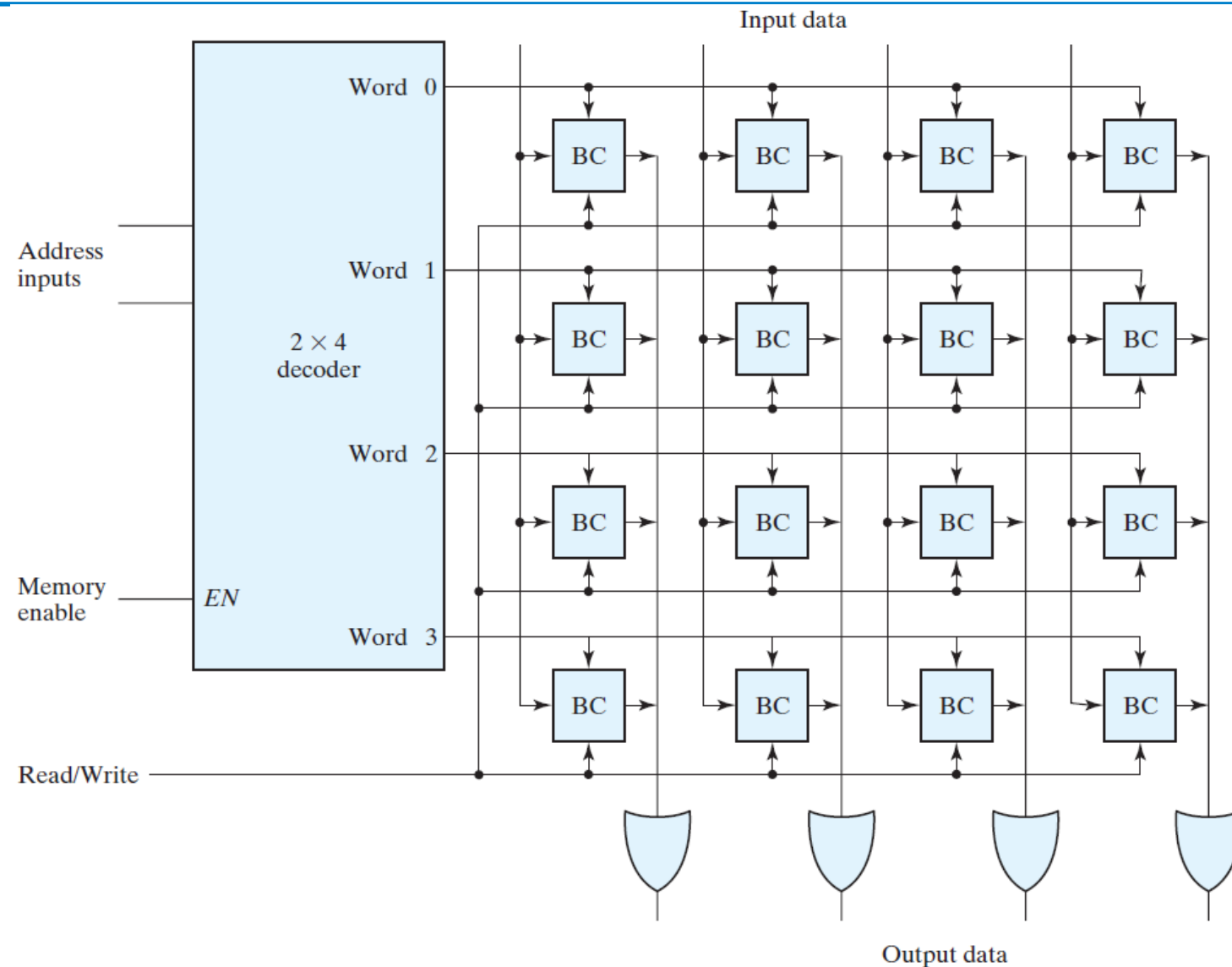
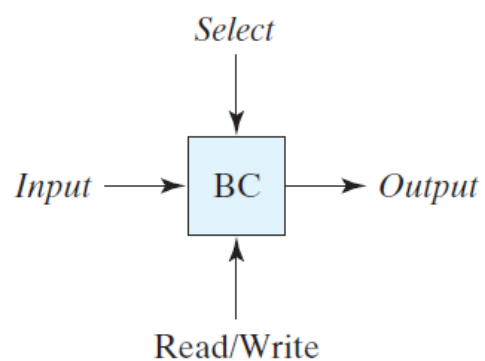
Memory Unit and Internal Construction

- A memory unit has two parts
 - The storage components (memory cell)
 - The decoding circuits to select the memory word
- A RAM of m (2^k) words and n bits per word
 - $m \times n$ binary storage cells
 - Decoding circuits to select individual words
 - k -to- 2^k decoder: address input to word line
 - Read/Write control
 - Input data/Output data



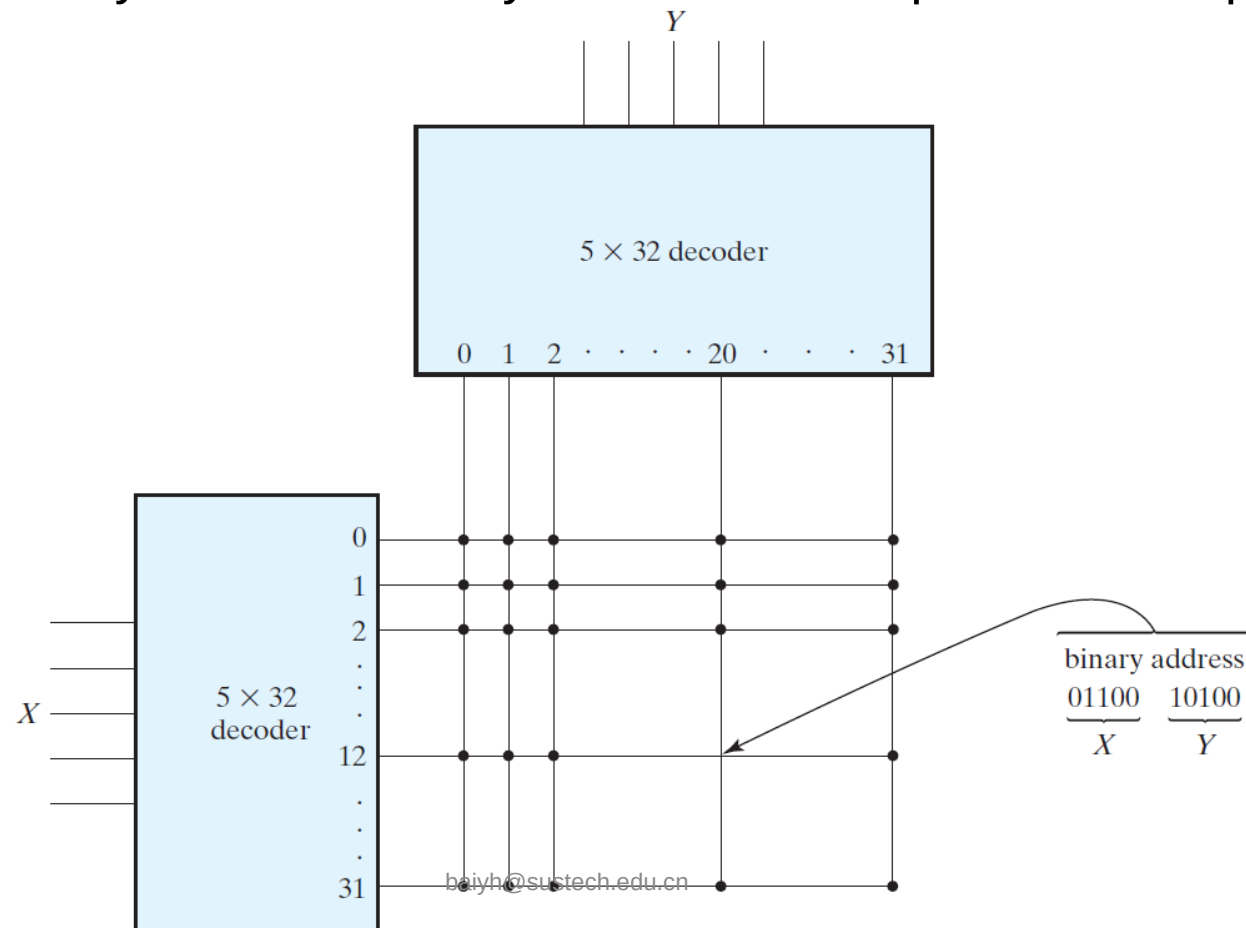
4x4 RAM

check the decoder
with enable input
in Chapter 4



Coincident Decoding

- A two-dimensional selection scheme
 - To reduce the complexity of the decoding circuits
 - To arrange the memory cells in an array that is close as possible to square

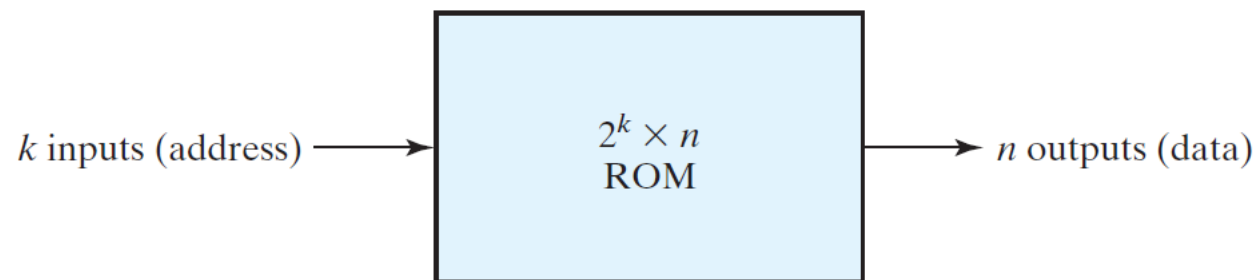


Coincident Decoding

- Example: 1k-word memory
 - A 10-to-1024 decoder
 - 1024 AND gates with 10 inputs per gate
 - Two 5-to-32 decoders
 - $2 \times (32 \text{ AND gates with 5 inputs per gate})$
 - Each word in the memory is selected by the coincidence between 1 of 32 rows and 1 of 32 columns for a total 1024 words
 - Two dimensional decoding structure reduces the circuit complexity and the cycle time of the memory

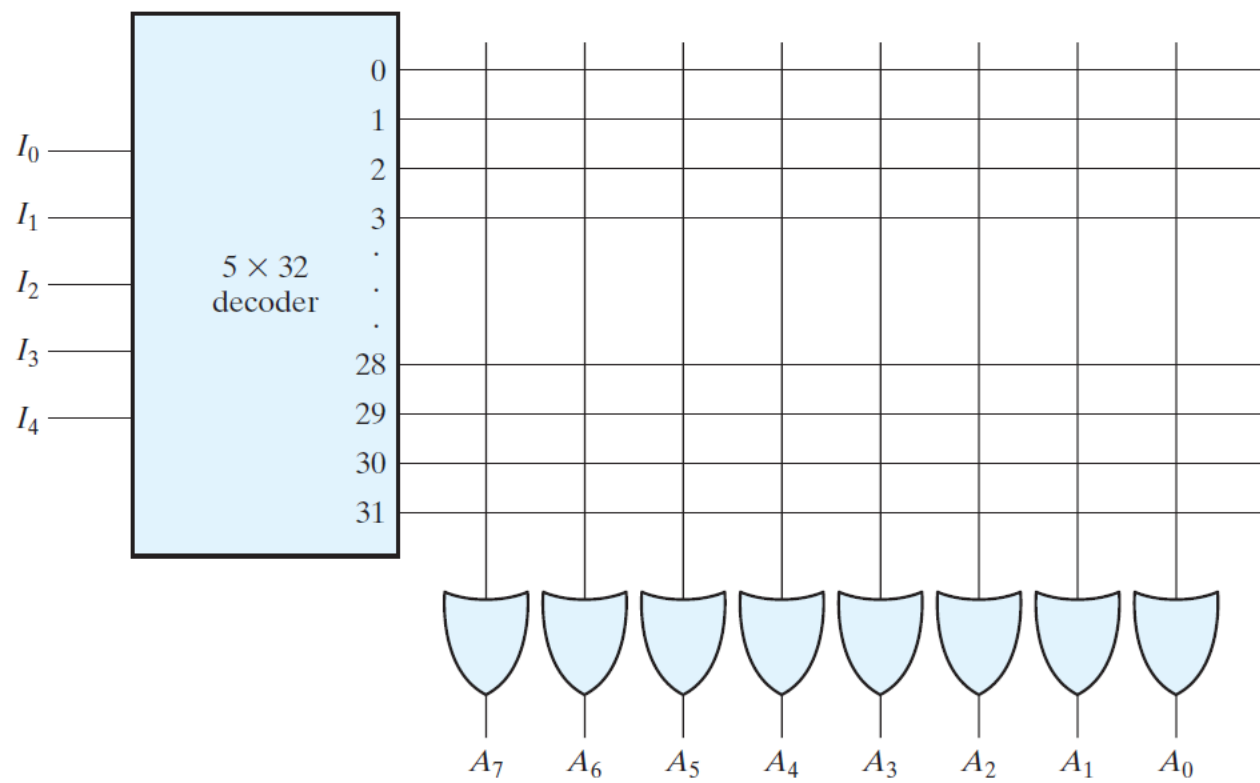
Read-Only Memory (ROM)

- ROM stores permanent binary information
- Once the pattern is established in the ROM, it stays within the unit, no matter power is on or off
- $2^k \times n$ ROM
 - k address input lines
 - enable input(s)



32x8 ROM

- 32 x 8 ROM ($2^5 \times 8$)
 - 5-to-32 decoder ($k=5$)
 - $32(2^5)$ outputs of the decoder are connected to each of the eight OR gates, which have $32(2^5)$ inputs
 - 32 x 8 internal programmable connections



ROM Programming

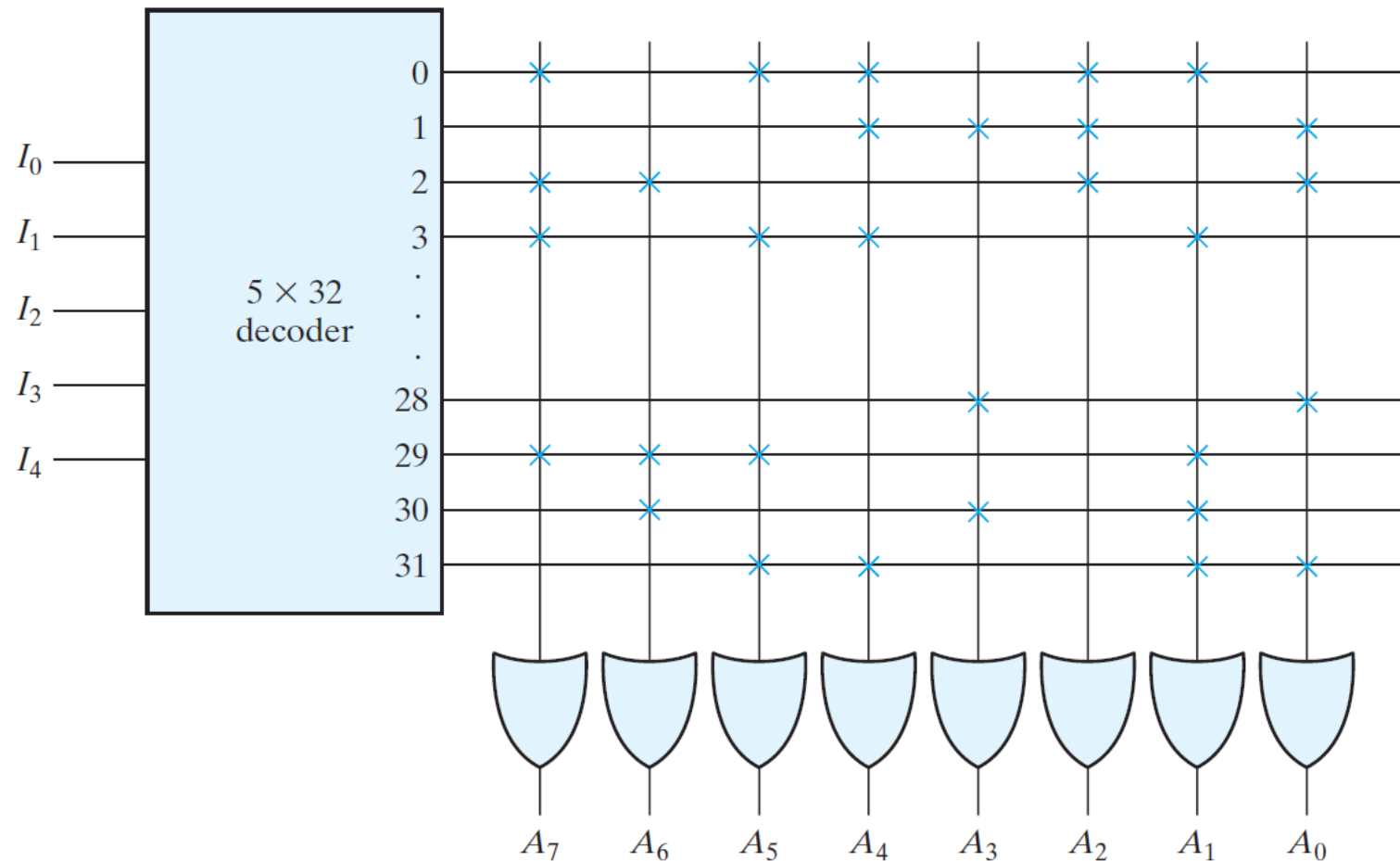
- PROM(Programmable ROM) with programmable interconnections
- close [1]: connected to high voltage
- open [0]: ground left
- A fuse that can be blown by applying a high voltage pulse

ROM Truth Table (Partial)

Inputs					Outputs							
I_4	I_3	I_2	I_1	I_0	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0
0	0	0	0	0	1	0	1	1	0	1	1	0
0	0	0	0	1	0	0	0	1	1	1	0	1
0	0	0	1	0	1	1	0	0	0	1	0	1
0	0	0	1	1	1	0	1	1	0	0	1	0
		$\vdots$							$\vdots$			
1	1	1	0	0	0	0	0	0	1	0	0	1
1	1	1	0	1	1	1	1	0	0	0	1	0
1	1	1	1	0	0	1	0	0	1	0	1	0
1	1	1	1	1	0	0	1	1	0	0	1	1

ROM Programming

- ROM programming according to ROM table using fusing



Combinational Circuit Implementation

- The internal operation of ROM can be interpreted in two ways
 - A memory contains a fixed pattern of stored words
 - A unit that implements a combinational circuit as sum of minterms.
- ROM: a decoder + OR gates
 - a Boolean function = sum of minterms
 - Ex. $A_7(I_4, I_3, I_2, I_1, I_0) = \sum(0, 2, 3, \dots, 29)$
 - For an n-input, m-output combinational circuit
 - $2^n \times m$ ROM
- Design Procedure
 - Determine the size of ROM
 - Obtain the programming truth table of ROM
 - The truth table = the fuse pattern

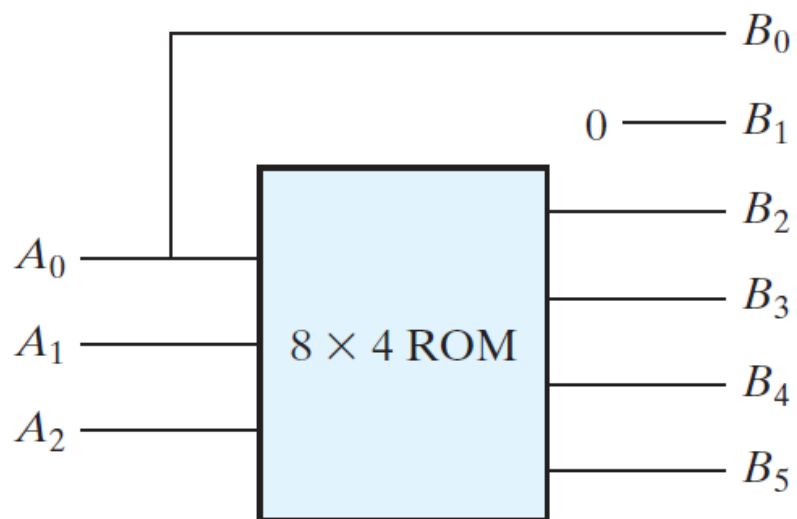
ROM Implementation

- A combinational circuit has 3 inputs, 6 outputs and the truth table, construct the circuit

Inputs			Outputs						Decimal
A_2	A_1	A_0	B_5	B_4	B_3	B_2	B_1	B_0	
0	0	0	0	0	0	0	0	0	0
0	0	1	0	0	0	0	0	1	1
0	1	0	0	0	0	1	0	0	4
0	1	1	0	0	1	0	0	1	9
1	0	0	0	1	0	0	0	0	16
1	0	1	0	1	1	0	0	1	25
1	1	0	1	0	0	1	0	0	36
1	1	1	1	1	0	0	0	1	49

$B_0=A_0$, $B_1=0$, leave only 8x4 ROM

ROM Implementation



(a) Block diagram

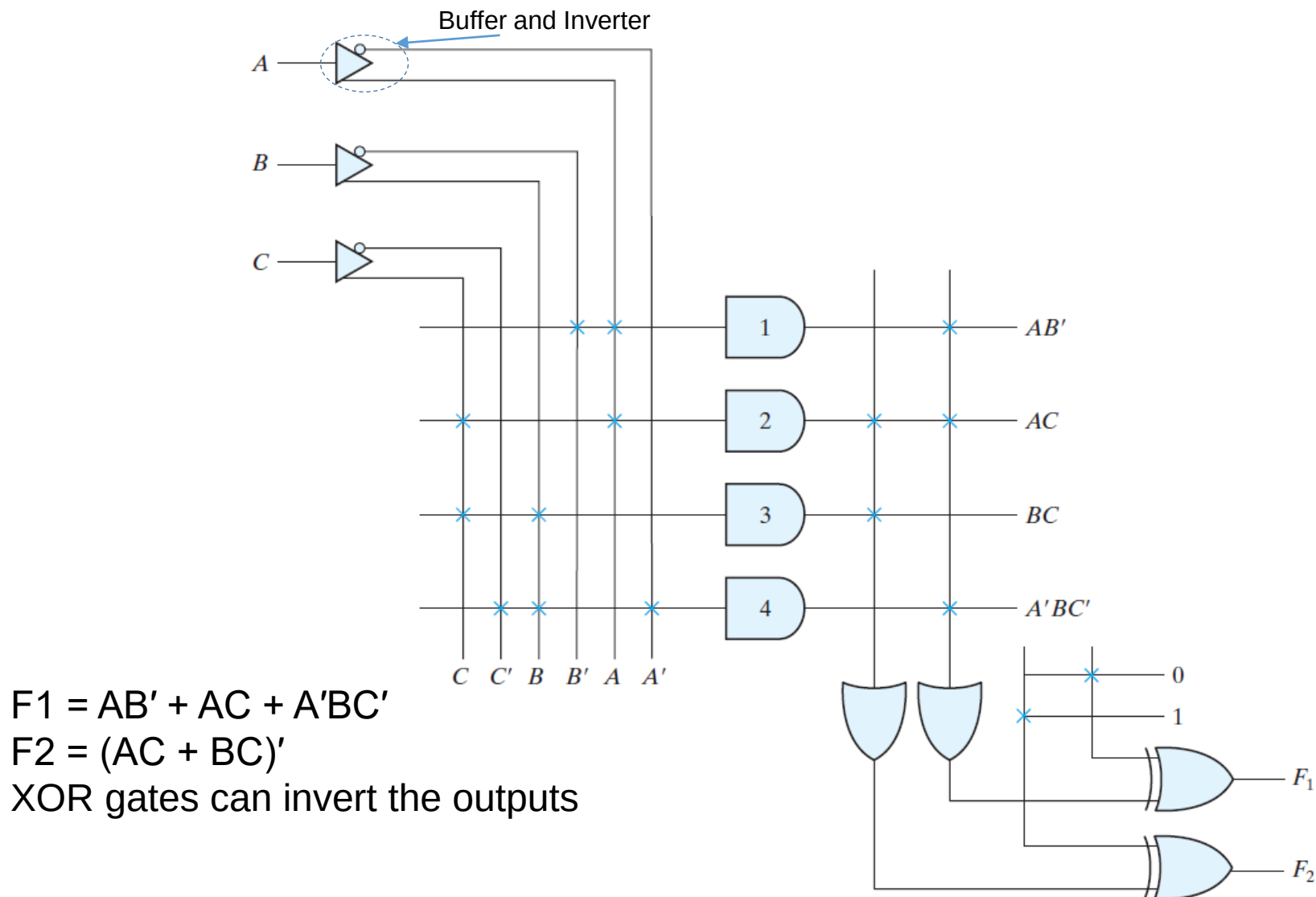
A_2	A_1	A_0	B_5	B_4	B_3	B_2
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	1	0	0	0	0	1
0	1	1	0	0	1	0
1	0	0	0	1	0	0
1	0	1	0	1	1	0
1	1	0	1	0	0	1
1	1	1	1	1	0	0

(b) ROM truth table

Programmable Logic Array

- PLA has two main parts
 - An array of programmable AND gates
 - Can generate any product terms of the inputs
 - An array of programmable OR gates
 - Can generate the sums of the products
- PLA is more flexible than PROM
 - PLA uses any combination of products while PROM uses sum of minterms
- PLA uses less circuits than PROM
 - Only the needed product terms are generated in PLA while all minterms must be generated in PROM

PLA



PLA

- Specify the fuse map
 - 1st col: list the product terms numerically
 - 2nd col: specify the required paths between inputs and AND gates
 - 3rd col: specify the required paths between AND gates and OR gates
 - (T)(C) stand for true or complement for programming XOR

PLA Programming Table

			Inputs			Outputs	
						(T)	(C)
	Product Term		A	B	C	F ₁	F ₂
AB'	1		1	0	—	1	—
AC	2		1	—	1	1	1
BC	3		—	1	1	—	1
$A'BC'$	4		0	1	0	1	—

Example

- Implement the following two Boolean functions with a PLA:
- $F_1(A, B, C) = \Sigma (0, 1, 2, 4)$
- $F_2(A, B, C) = \Sigma (0, 5, 6, 7)$
 - Both true and complement of the function should be simplified to check

$A \backslash BC$	00	01	11	10
0	1	1	1	1
1	1	0	1	0

$$F_1' = AB + AC + BC$$

$$F_1 = (AB + AC + BC)'$$

$A \backslash BC$	00	01	11	10
0	1	0	0	0
1	0	1	1	1

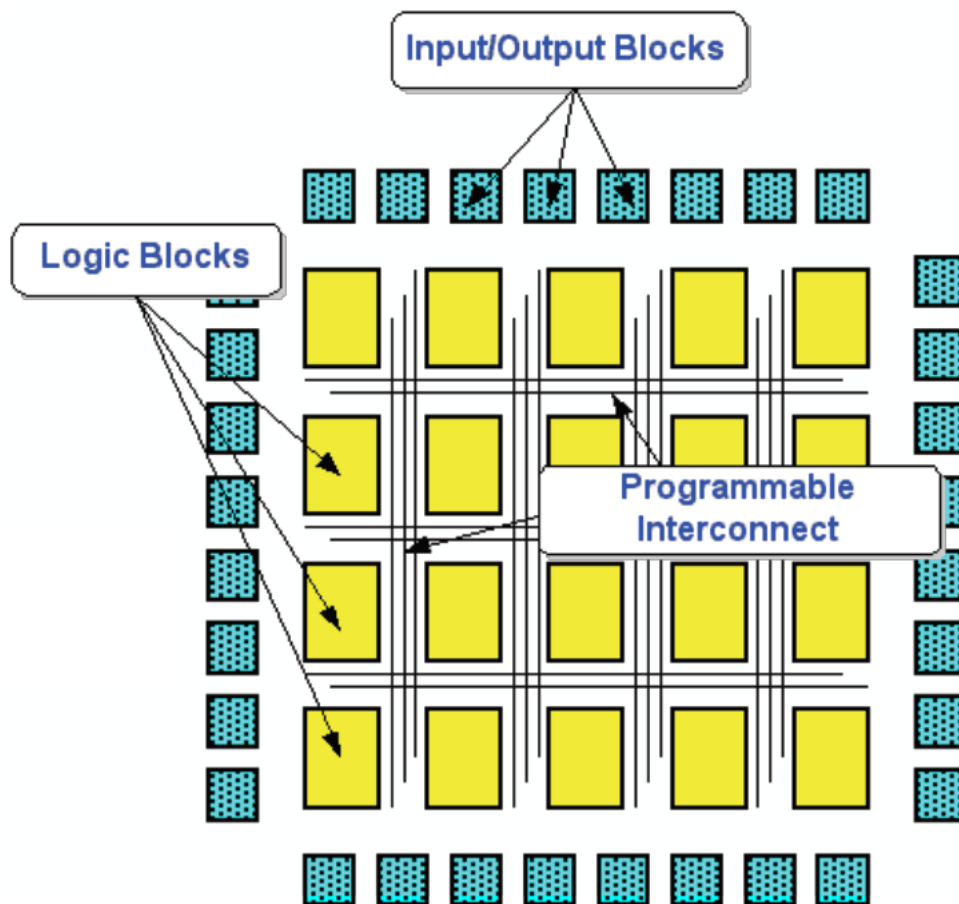
$$F_2 = AB + AC + A'B'C'$$

PLA programming table

	Product term	Inputs			Outputs	
		A	B	C	(C) F_1	(T) F_2
AB	1	1	1	–	1	1
AC	2	1	–	1	1	1
BC	3	–	1	1	1	–
$A'B'C'$	4	0	0	0	–	1

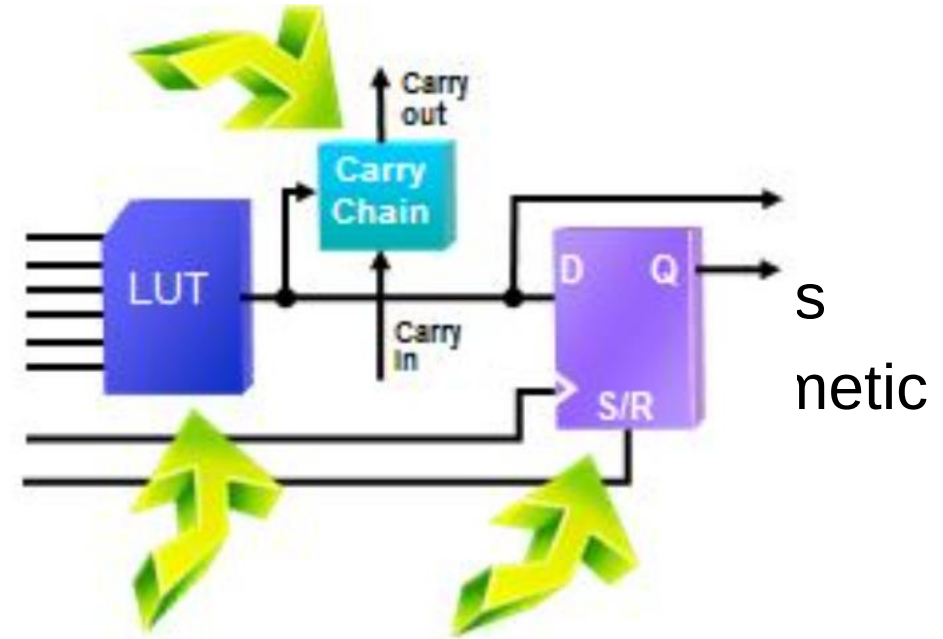
FPGA

- What is in the array? All sorts of stuff...
 - I/O Cells
 - Logic Cells
 - Memories
 - Microprocessors
 - Clock Management
 - High Speed I/O Transceivers
 - Programmable routing

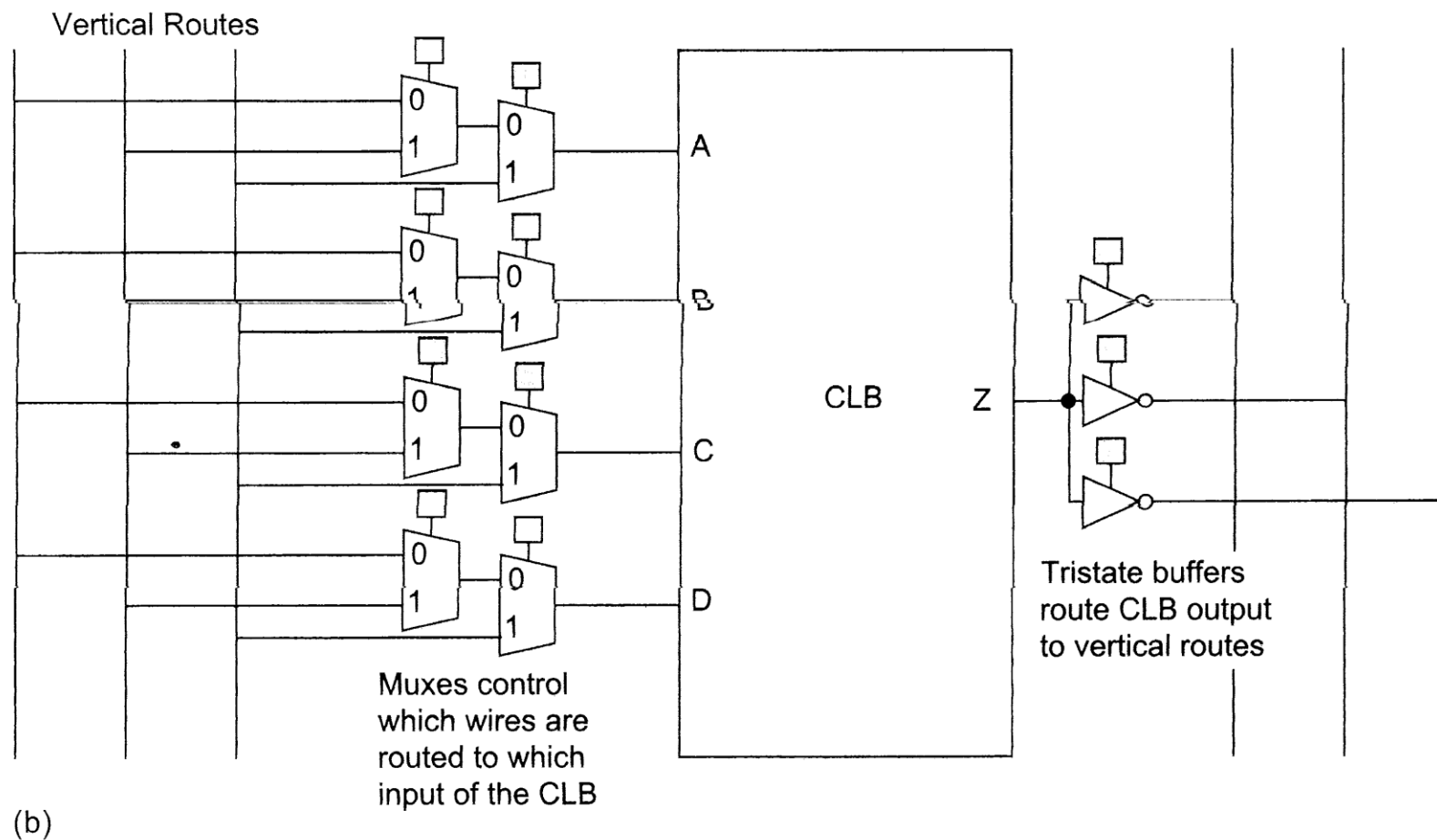


Simple FPGA Logic Cell

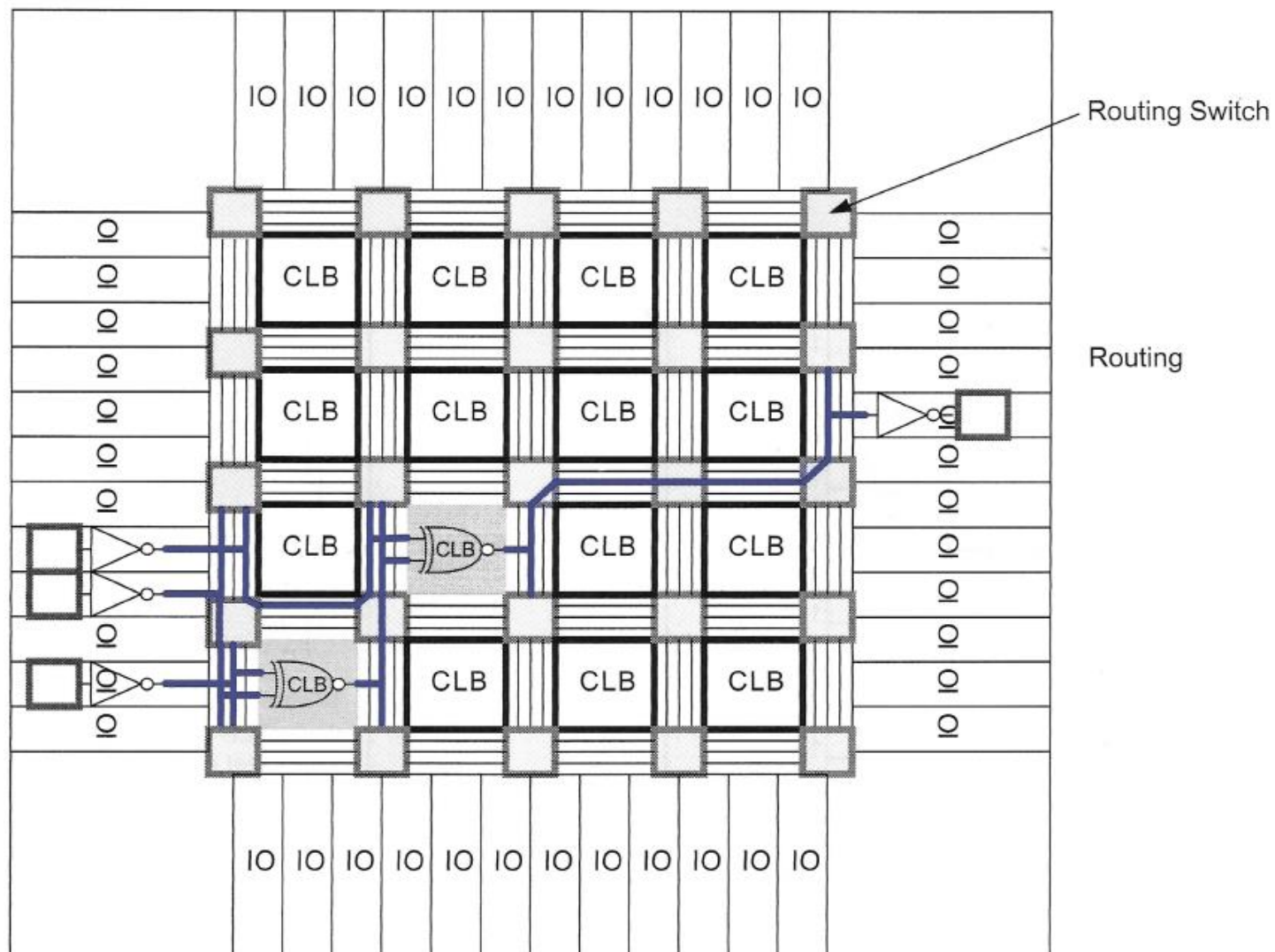
- Logic cells include
 - Combinatorial logic, arithmetic logic, and a register
- Combinatorial logic is implemented using LUTs
- Register functions can include latches, J/K flip-flops, and D flip-flops
- Arithmetic logic is a dedicated carry chain for ripple-carry operations



Simple FPGA Routing Cell



Routed FPGA



JKFF FPGA implementation

Characteristic table			
JK Flip-Flop			
J	K	Q(t + 1)	
0	0	Q(t)	No change
0	1	0	Reset
1	0	1	Set
1	1	Q'(t)	Complement

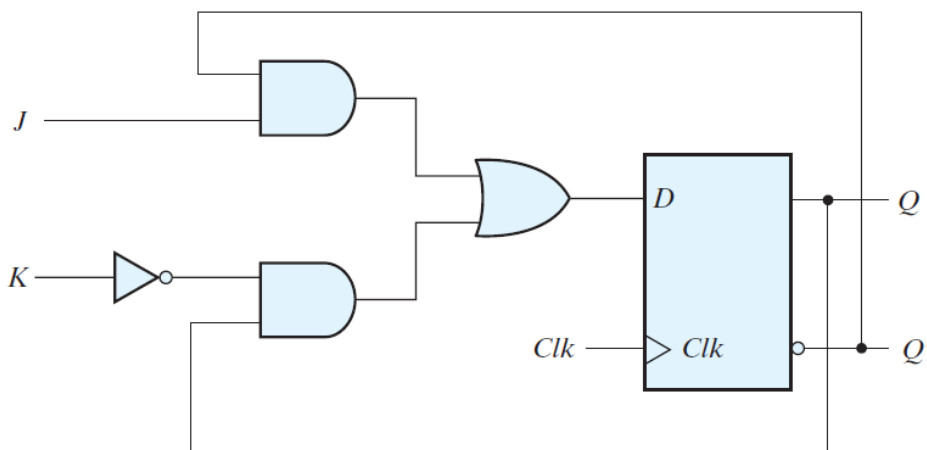
Derive the Truth table



J	K	Q(t)	Q(t+1)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Characteristic equation of JKFF

$$Q(t+1) = JQ(t)' + K'Q(t)$$



```

23 module jkff(
24     input clk, J, K,
25     output reg Q,
26     output nQ
27 );
28     assign nQ = ~Q;
29     always @(posedge clk)
30     case ({J, K})
31         2'b10: Q <= 1'b1;
32         2'b01: Q <= 1'b0;
33         2'b11: Q <= nQ;
34         2'b00: Q <= Q;
35     endcase
36 endmodule

```

JKFF FPGA implementation

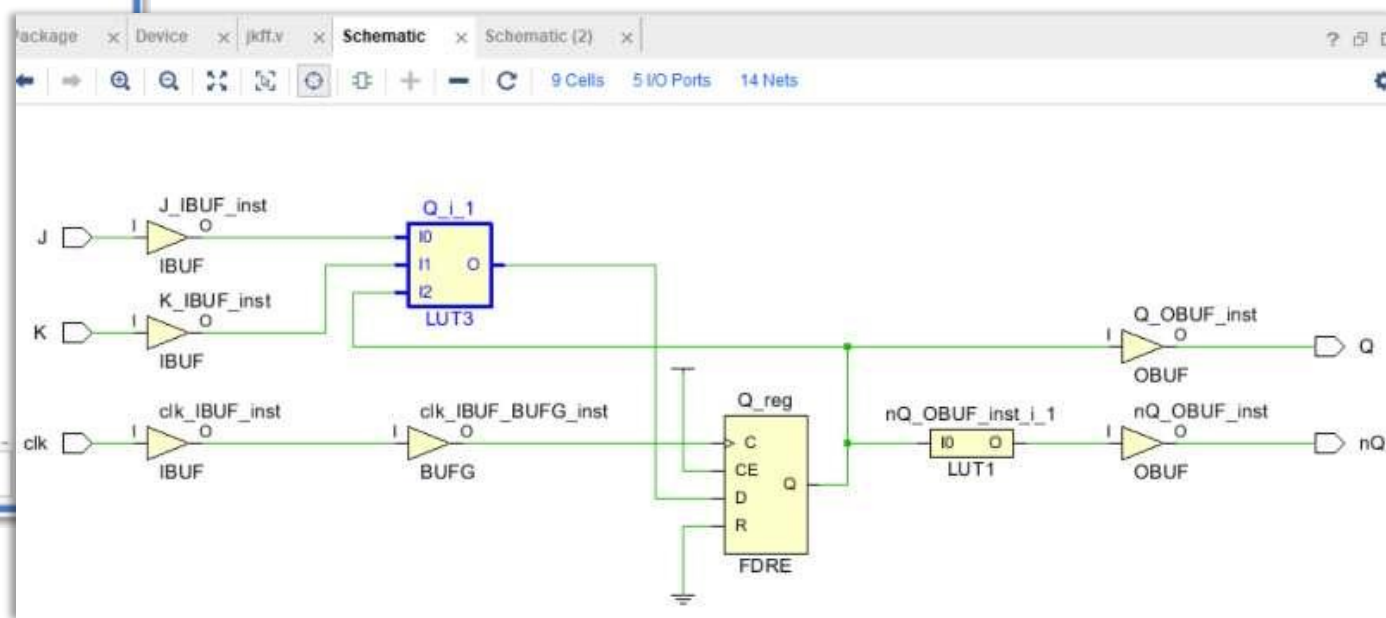
Cell Properties

Q_i_1

I2	I1	I0	O=I0 & !I2 + !I1 & I2
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

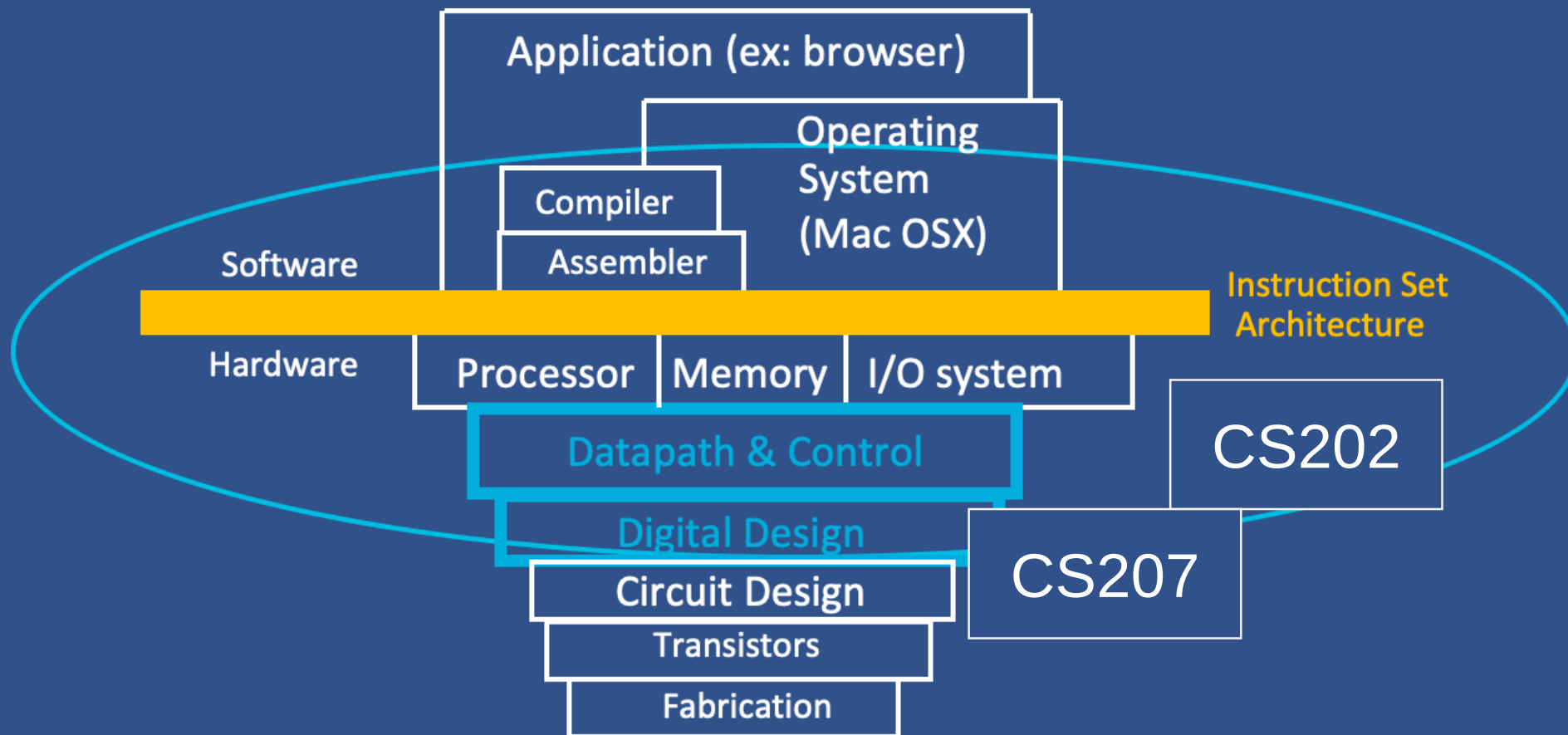
Edit LUT Equation...

General Properties Power Nets Cell Pins Truth Table



Follow-up course of Digital Logic

- CS202 Computer Organization



Abstractions and Instruction Set Architecture

High Level Language
Program (e.g., C)

```
temp = v[k];
v[k] = v[k+1];
v[k+1] = temp;
```

| *Compiler*

Assembly Language
Program (e.g., RISC-V)

```
lw    x3, 0(x10)
lw    x4, 4(x10)
sw    x4, 0(x10)
sw    x3, 4(x10)
```

| *Assembler*

Machine Language
Program (RISC-V)

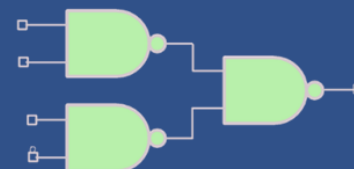
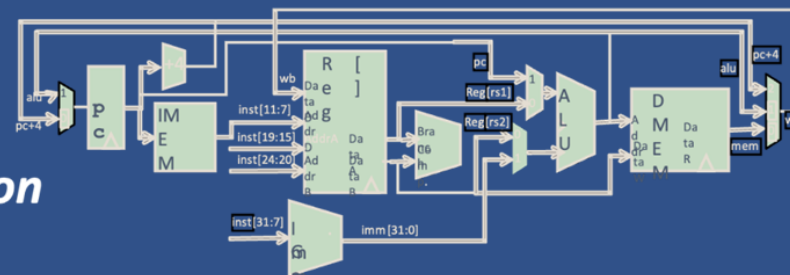
```
1000 1101 1110 0010 0000 0000 0000 0000
1000 1110 0001 0000 0000 0000 0000 0100
1010 1110 0001 0010 0000 0000 0000 0000
1010 1101 1110 0010 0000 0000 0000 0100
```

Anything can be
a **number**—data,
instructions, etc.

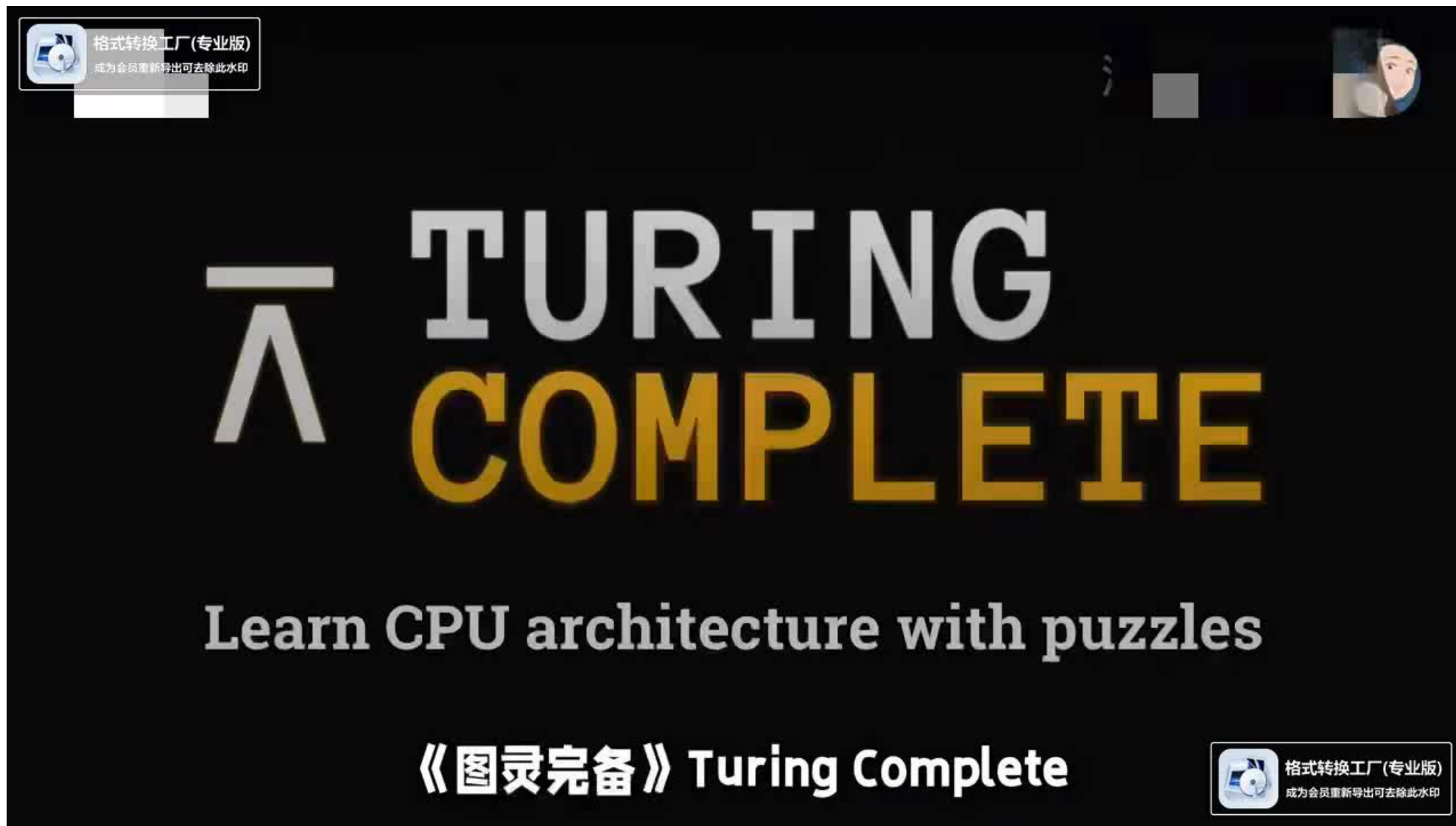
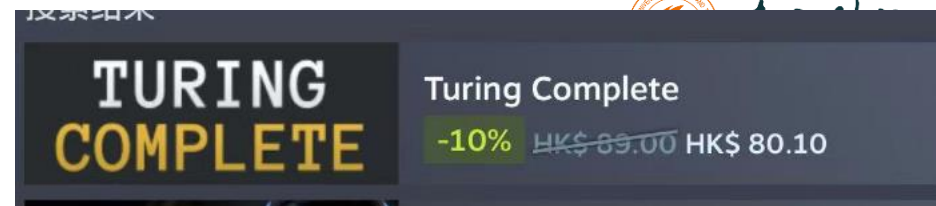
Hardware Architecture Description
(e.g., block diagrams)

| *Architecture Implementation*

Logic Circuit Description
(Circuit Schematic Diagrams)



Steam Game: Turing Complete



系统能力评测

- <https://test.xtnl.org.cn/aplicationGuide.html>

计算机硬件系统开发能力评测

考核内容

- 考察对计算机系统硬件基础知识的掌握程度。

考核方式

- 均为理论知识考察，不考察实践能力。

题目形式

- 单项选择题、多项选择题。

你需要掌握的内容

- 计算机的基本组成（运算器、存储器、输入输出设备）；
- 不同数制之间的转换、常见数据表示和存储方式；
- 基本的算术和逻辑运算原理；
- 指令集的基本概念和作用；
- 存储层次结构（主存、Cache、虚拟存储器）的基本思想；
- 中断和异常的基本概念；
- I/O 和总线的常见方式和原理；

理论一级：计算机系统硬件认知能力（RISC-V）

理论一级：计算机系统硬件认知能力（MIPS）

理论一级：计算机系统硬件认知能力（RISC-V）

理论二级：计算机系统硬件应用能力（MIPS）

理论二级：计算机系统硬件应用能力（RISC-V）

实践一级：计算机系统硬件部件设计能力

实践二级：计算机系统硬件综合设计与集成能力

系统能力

首页 备考指南 新闻 登录 | 注册

系统能力培养人才评价平台

登录/注册

关于评测

操作指南

设备检测

我的报名

点击查看报名详情

立即前往

证书查询

点击查看评测成绩/证书

立即前往

公告

- 关于评测成绩异常的说明 2025-12-05
- 【重要通知】12月5日起计算机硬件系统开发能力评测成绩... 2025-12-04
- 【正式评测重要事项提醒】首次系统能力人才评测明天9:00... 2025-11-14
- 【测前必看】模拟/正式评测全流程视频讲解！ 2025-11-12

“龙芯杯”全国大学生计算机系统能力培养大赛 NSCSCC

- NSCSCC（大赛官网：nscscc.com）
 - 教育部白名单赛事
 - 南科大计算机系2025推免-学科竞赛加分

• 赛道	赛道	参赛人数	指令集	侧重点
	个人赛	1	MIPS / LoongArch	指令功能实现+性能优化
	团队赛	2~4	LoongArch	启动linux+性能优化

- 赛程（以2025年为例）

